Neuro-symbolic approaches to the SAT problem

Intelligent Systems for Pattern Recognition

Reinforcement Learning

University of Pisa

survey with coding project

Donato Meoli

The SAT problem

- **SAT**: given a Boolean formula in CNF $\varphi = \bigwedge_i \left(\bigvee_j \ell_{ij} \right)$, is there an assignment of the variables that makes it true? First problem proven NP-complete; the engine of verification, planning, synthesis.
- Modern solvers are **CDCL**: *decide* a variable, *unit-propagate*, on a *conflict* learn a clause and *backtrack*.
- The **branching heuristic** (which variable to decide next, e.g. VSIDS) drives performance — yet it makes poor choices during its warm-up phase, when activity scores are still uninformative.
- *This project*: **learn the branching heuristic** with two neuro-symbolic methods, modernise them to PyTorch, study **where graph attention helps**, and reproduce the published results.

Roadmap

Two ways to learn a SAT heuristic:

- **Graph-Q-SAT** — value-based RL (DQN) over a *graph* neural network on the CNF graph. Size-agnostic.
- **AlphaZeroSAT** — self-play + MCTS over a *CNN* policy/value net on the clause $\times$ variable matrix. Fixed size.

Our contributions:

- port both from TensorFlow 1.x + GSL to one PyTorch / PyTorch-Geometric stack; reproduce the published numbers *exactly*;
- **GAT-Q-SAT** and **GTv2-Q-SAT**: attention-augmented successors;
- **Model3Attn**: a self-attention variant of AlphaZeroSAT;
- **restricted heuristics** for online speed-ups.

Plan: background on the CNF graph $\rightarrow$ Graph-Q-SAT & GAT $\rightarrow$ AlphaZeroSAT $\rightarrow$ implementation $\rightarrow$ results.

- The CNF as a graph
- Message Passing Neural Networks (MPNNs)
- NeuroSAT — the seminal graph model for SAT

(ISPR – survey)

Gilmer et al., *Neural Message Passing for Quantum Chemistry*.
Selsam et al., *Learning a SAT Solver from Single-Bit Supervision*.

Background: learning on the CNF graph

The CNF as a graph

A CNF is naturally a **bipartite graph**:

- a node for every **variable** (or literal) and every **clause**;
- an edge between a literal and each clause it occurs in (edge feature = **polarity**); for the literal view, an edge between complementary literals $x, \bar{x}$.

This representation is **permutation-invariant** and **size-agnostic**: the same network handles formulas of any size — the property the fixed-size CNN lacks. It is the substrate shared by NeuroSAT and Graph-Q-SAT.

$$\varphi = (x_1 \vee \bar{x}_2) \wedge (\bar{x}_1 \vee x_2 \vee x_3)$$

variables	clauses
x_1	
x_2	c_1
x_3	c_2

edges carry the literal polarity $\{+, -\}$.

Message Passing Neural Networks (MPNNs)

A general framework for learning on graphs (Gilmer et al., 2017). Each node v holds a hidden state h_v^t; for T rounds it exchanges messages with its neighbours $\mathcal{N}(v)$:

$$\textbf{message:} \quad m_v^{t+1} = \sum_{w \in \mathcal{N}(v)} M_t\big(h_v^t,\, h_w^t,\, e_{vw}\big)$$

$$\textbf{update:} \quad h_v^{t+1} = U_t\big(h_v^t,\, m_v^{t+1}\big)$$

$$\textbf{readout:} \quad \hat{y} = R\big(\{\, h_v^T : v \in G \,\}\big)$$

- M_t (message), U_t (update, often a GRU/LSTM) and R (readout) are learned and *shared across nodes and rounds*.
- GNNs, gated GNNs, GCN, GAT and the encode–process–decode network are all instances of this template.

NeuroSAT — a SAT solver from single-bit supervision

Literal/clause embeddings $L^{(t)} \in \mathbb{R}^{2n \times d}$, $C^{(t)} \in \mathbb{R}^{m \times d}$; bipartite adjacency M; MLPs $L_{\mathsf{msg}}, C_{\mathsf{msg}}, L_{\mathsf{vote}}$; LSTM updates L_u, C_u. One round:

$$(C^{(t+1)}, C_h^{(t+1)}) \leftarrow C_u\big([\, C_h^{(t)},\ M^\top L_{\mathsf{msg}}(L^{(t)})\,]\big)$$
$$(L^{(t+1)}, L_h^{(t+1)}) \leftarrow L_u\big([\, L_h^{(t)},\ F(L^{(t)}),\ M\, C_{\mathsf{msg}}(C^{(t+1)})\,]\big)$$

after T rounds, vote and average:

$$L_*^{(T)} \leftarrow L_{\mathsf{vote}}(L^{(T)}), \qquad y^{(T)} \leftarrow \operatorname{mean}\big(L_*^{(T)}\big).$$

F swaps each literal with its complement.

- Trained *only* as a SAT/UNSAT classifier (one bit per formula).
- Yet it **learns to find satisfying assignments**: it guesses UNSAT with low confidence until a solution crystallises in the literal votes.
- Establishes the **graph view** of a CNF — the foundation for Graph-Q-SAT.
- *Limitation*: a standalone solver, not competitive in wall-clock with CDCL.

- Reinforcement learning for branching
- Background
  - Deep Q-Learning (DQN)
  - Graph Networks
- Model — encode–process–decode
- Graph Attention (GAT, GATv2)
- Results

(ISPR + RL – survey with extra coding)

Kurin et al., *Can Q-Learning with Graph Networks Learn a Generalizable Branching Heuristic for a SAT Solver?*

Graph-Q-SAT

&

GAT-Q-SAT

Branching as a Markov Decision Process

Graph-Q-SAT turns the CDCL search into an MDP $(\mathcal{S}, \mathcal{A}, P, r, \gamma)$:

- **state** s: the current CNF as a bipartite graph (variable/clause nodes, polarity edges, a global attribute);
- **action** a: assign an unassigned variable True/False — one decision;
- **reward** $r = -p$ at every non-terminal step, 0 at termination $\Rightarrow$ maximising the return *minimises the number of decisions*;
- **return** $G_t = \sum_{k \geq 0} \gamma^k \, r_{t+k}$.

Goal: learn the action-value (*quality*) function

$$Q^\star(s, a) = \mathbb{E}\big[\, r + \gamma \max_{a'} Q^\star(s', a') \,\big] \qquad \text{(Bellman optimality)}.$$

The branching heuristic is then $\pi(s) = \arg\max_a Q^\star(s, a)$ over the variable nodes.

Deep Q-Learning (DQN)

Approximate $Q^\star$ with a network Q_θ, fitted by minimising the temporal-difference error over a replay buffer $\mathcal{D}$:

$$\mathcal{L}(\theta) = \mathbb{E}_{(s,a,r,s')\sim\mathcal{D}}\left[\Big(\underbrace{r + \gamma \max_{a'} Q_{\theta^-}(s', a')}_{\text{TD target}} - Q_\theta(s, a)\Big)^2\right]$$

- **target network** θ^- (a slow copy of θ) stabilises the target;
- **experience replay** $\mathcal{D}$ breaks temporal correlations;
- ε-**greedy** exploration: random action w.p. ε, else $\arg\max_a Q_\theta$.

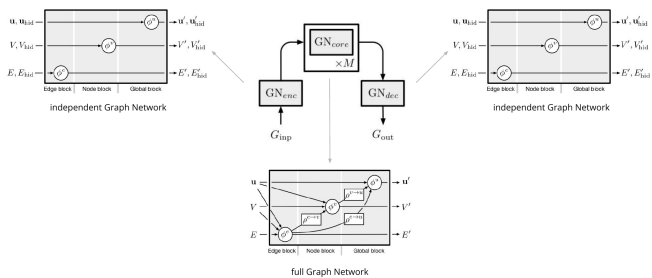
Here the action set is *variable-sized* (one per unassigned variable), so Q_θ must be a **graph** network that scores each variable node.

Model — the encode–process–decode graph network

A graph-network block (Battaglia et al.) updates edges, then nodes, then the global:

$$e'_k = \phi^e\left(e_k, v_{r_k}, v_{s_k}, u\right)$$
$$v'_i = \phi^v\left(\rho^{e\to v}(\{e'_k\}), v_i, u\right)$$
$$u' = \phi^u\left(\rho^{e\to u}(\{e'_k\}), \rho^{v\to u}(\{v'_i\}), u\right)$$

ϕ are MLPs; ρ are permutation-invariant aggregations (sum). **Encode** (independent) → **core** ($\times M$ message-passing steps, shared) → **decode**; the per-variable Q-values come out of the decoder.



encode–process–decode: $\mathrm{GN}_{enc} \to \mathrm{GN}_{core} \times M \to \mathrm{GN}_{dec}$.

Configuration & metric

How the model is configured:

- node features: 2-D one-hot (variable / clause); edge feature: polarity; global: empty.
- encoder/core/decoder hidden size 64; M core message-passing steps (a key knob at test time);
- DQN: Adam, target-net sync, prioritised replay, $\gamma \approx 0.99$.

Metric — MRIR:

$$\text{MRIR} = \text{median}\, \frac{\#\text{iters}_{\text{MiniSat}}}{\#\text{iters}_{\text{model}}}$$

Median Relative Iteration Reduction: > 1 means *fewer* CDCL iterations than MiniSat's VSIDS. Reported as a function of the test-time decision budget (the cap).

Improvement idea: graph attention

- In the core block, a node aggregates *all* neighbours with equal weight ($\rho = \text{sum}$). But not all neighbouring clauses/variables are equally relevant to a branching decision.
- **Idea**: replace the uniform aggregation with a **learned, edge-aware attention** over neighbours, so the network can encode more abstract dependencies in the SAT graph.
- This is the *non-local* graph-network variant — a **Graph Attention Network (GAT)** inside the core.

Veličković et al., *Graph Attention Networks* (ICLR 2018); Brody et al., *How Attentive are GATs?* (ICLR 2022).

Graph Attention Networks (GAT)

For node i with neighbours $\mathcal{N}_i$ and a shared linear map W, attention weights a message by relevance:

$$\begin{aligned}
\text{score:} \quad & e_{ij} = \mathsf{LeakyReLU}\big(\mathbf{a}^\top [\, Wh_i \,\|\, Wh_j \,]\big) \\
\text{normalise:} \quad & \alpha_{ij} = \mathrm{softmax}_j(e_{ij}) = \frac{\exp(e_{ij})}{\sum_{k \in \mathcal{N}_i} \exp(e_{ik})} \\
\text{aggregate:} \quad & h'_i = \sigma\Big(\textstyle\sum_{j \in \mathcal{N}_i} \alpha_{ij}\, Wh_j \Big)
\end{aligned}$$

- **multi-head**: K independent attentions are concatenated (or averaged) for stability and capacity;
- edges carry the literal polarity, so attention is **edge-featured**.

GAT-Q-SAT & GTv2-Q-SAT — our attention successors

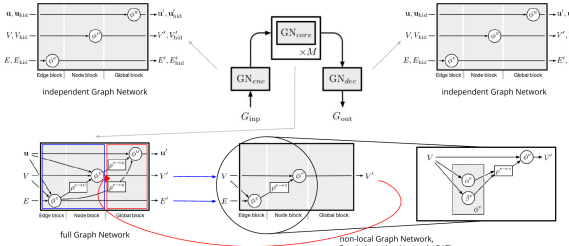
GAT-Q-SAT: inject two edge-aware multi-head `GATConv` layers into the core block (the non-local block, right).

GTv2-Q-SAT (further successor, inspired by NeuroBack): a *pre-norm Transformer block* on **GATv2**, which makes attention *dynamic* by moving the nonlinearity before $\mathbf{a}$:

$$e_{ij} = \mathbf{a}^\top \,\mathrm{LeakyReLU}\big(W[\,h_i \,\|\, h_j\,]\big)$$

$$h' = \mathrm{LN}\big(h + \mathrm{GATv2}(h) + \mathrm{FFN}(h)\big)$$

lineage: **Graph-Q-SAT** → **GAT-Q-SAT** → **GTv2-Q-SAT**.



the core becomes a *non-local* GN block — a GAT with edge attention α^e, β^e.

- Search-based RL for branching
- Background
  - Monte Carlo Tree Search (MCTS)
  - AlphaZero self-play
- Model — CNN policy/value net
- Results

(ISPR + RL – survey with coding)

Wang & Rompf, *From Gameplay to Symbolic Reasoning: Learning SAT Solver Heuristics in the Style of Alpha(Go) Zero*.

AlphaZeroSAT

Monte Carlo Tree Search (MCTS)

Treat solving as a game: each node is a partial assignment, each move a branching literal. From the root, repeat four steps:

1. **select** the child maximising the PUCT score

$$a_t = \arg\max_a \Big(Q(s,a) + \underbrace{c_{\text{puct}}\, P(s,a) \frac{\sqrt{\sum_b N(s,b)}}{1+N(s,a)}}_{U(s,a)} \Big);$$

2. **expand** a leaf and evaluate it with the network $(P, v) = f_\theta(s)$;
3. **simulate** / bootstrap from the value v;
4. **back-up**: $N \mathrel{+}= 1$, $W \mathrel{+}= v$, $Q = W/N$.

The search yields an improved policy from the visit counts: $\pi(a \mid s) \propto N(s,a)^{1/\tau}$.

AlphaZero — self-play policy iteration

A single network $(p, v) = f_\theta(s)$ outputs a **policy** p over moves and a **value** $v \in [-1, 1]$ (probability of solving from s).

- **Self-play**: MCTS, guided by f_θ, produces a stronger policy π; play it to the end to obtain an outcome z.
- **Learn**: regress $v \to z$ and $p \to \pi$ — MCTS is the policy-improvement operator, the network is the policy-evaluation operator:

$$\ell = \underbrace{(z - v)^2}_{\text{value}} \quad \underbrace{-\pi^\top \log p}_{\text{policy (cross-entropy)}} \quad + \quad \underbrace{c\|\theta\|^2}_{\text{regularisation}} \quad .$$

- No human data, no heuristics: the heuristic *emerges* from self-play.

AlphaZeroSAT — the model

State: the CNF as a fixed-size **clauses** × **variables** matrix (entries $\{+1, 0, -1\}$ by literal polarity, as image channels).

Network: a CNN trunk (conv $32@8\times8$, $64@4\times4$, $64@3\times3$) then two heads:

$$\text{policy } p = \text{softmax}\big(\text{logits} + (\text{mask} - 1)\cdot\infty\big) \in \Delta^{2n}$$
$$\text{value } v = \tanh(\cdot) \in [-1, 1]$$

invalid literals (variables absent from remaining clauses) are masked to $-\infty$.

Model1/2/3 differ in the heads (fc vs convolutional); **Model3Attn** (this work) adds a multi-head **self-attention** block over the conv feature map — the Alpha(Go)Zero analogue of GAT-Q-SAT.

Env: MiniSat is extended into an MCTS *game* supporting hypothetical simulations.

Metric: mean branching *decisions* to solve (lower is better), reproducing the paper's Fig. 2.

Limitation: the fixed-size matrix means the CNN **cannot generalise across problem sizes** — precisely the motivation for the graph methods.

- Modernisation & porting
- Restricted heuristics (online speed-ups)
- Reproduction (semantic non-regression)
- Where attention helps

(the coding project)

Implementation & Results

Modernisation — one self-contained PyTorch stack

- Ported **both** methods from **TensorFlow 1.x + GSL** to latest `torch`, `torch-geometric`, `numpy`$\geq$2, `gymnasium` — no `torch-scatter/sparse`, no GSL.
- Graph-Q-SAT: `scatter` drop-in, gym-robust env construction, a version-agnostic checkpoint loader (bridging the `GATConv` parameter renames across PyG versions).
- AlphaZeroSAT: re-implemented the CNN + AlphaZero loss in PyTorch; **removed GSL** (Dirichlet noise via C++11 `<random>`); trains end-to-end on GPU.
- Datasets unified into one shared `data/` hub; both engines kept independent and runnable on Colab with Drive checkpointing.

Restricted heuristics — trading decisions for speed

Graph-Q-SAT cuts CDCL *iterations* but the GNN forward at every decision can lose on *wall-clock*. Three online tricks (Shirokikh et al. 2023), all added at eval time:

- **early release** (`-release_after N`): guide the first N decisions, then hand control back to MiniSat's VSIDS;
- **action pool** (`-action_pool_size K`): one GNN forward $\rightarrow$ top-K actions executed without re-running the net;
- **Q-value warm-start** (`-warmstart_release`): seed VSIDS activities from the root-state Q-values, $\text{activity}(x) = -1/\max\big(Q(x,\mathsf{T}), Q(x,\mathsf{F})\big)$, then run pure VSIDS.

The model matters most at the cold start, where VSIDS is still blind.

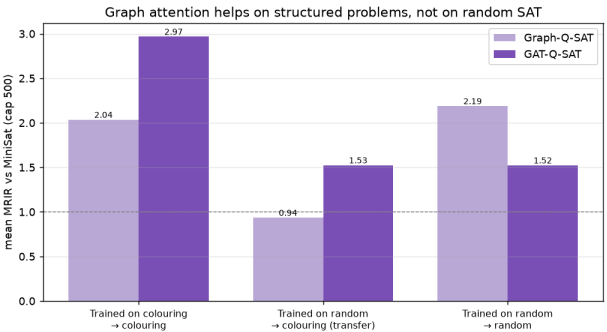
Reproduction (semantic non-regression)

- Re-ran the *original trained checkpoints* on the modern stack (latest torch / PyG / numpy$\geq$2 / gymnasium).
- MRIR matches the committed logs **exactly**:

Model	MRIR (median)	committed
Graph-Q-SAT	1.833	1.833
GAT-Q-SAT	1.667	1.667

- Drop-in changes only (scatter, env construction, GATConv key remap) — **semantics preserved**. Figures regenerated from the logs (no Sheets).

Results — the central finding

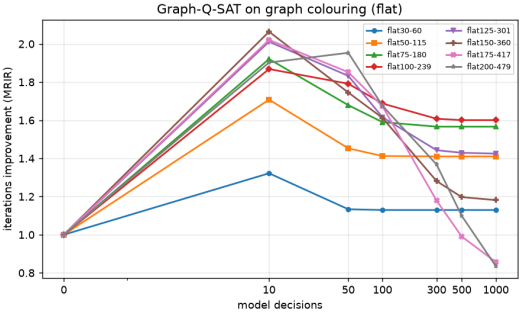


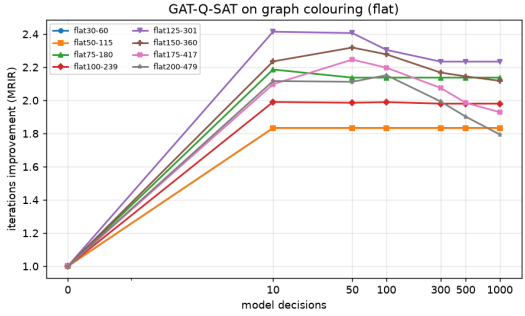
Clean attention on/off ablation, matched training sets (mean MRIR, cap 500):

- **Colouring** (in-dist): GAT **2.97** vs 2.04.
- **Transfer** random→colouring: GAT **1.53** vs 0.94 (Graph-Q-SAT < 1, i.e. worse than MiniSat!).
- **Random** 3-SAT: attention *does not* help (1.52 vs 2.19).

Attention helps on structured problems, not on uniform-random SAT.

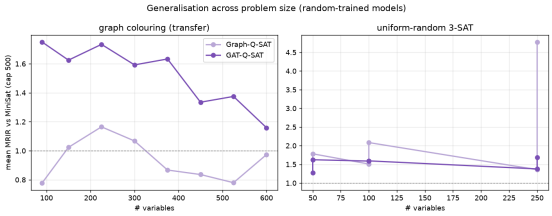
Results — MRIR curves on graph colouring





Iteration improvement vs the decision budget, one line per `flat` family — **GAT-Q-SAT lifts the whole curve** on every size.

Results — attention helps more on *larger* structured problems



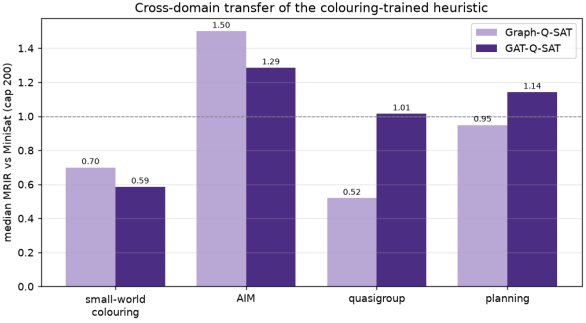
Random-trained: MRIR vs problem size (cap 500).

In-distribution graph colouring, MRIR per size:

set	Graph	GAT	Δ
flat30-60	1.83	2.00	+0.17
flat125-301	2.55	3.44	+0.88
flat150-360	1.92	3.76	+1.85
flat200-479	1.35	3.39	**+2.04**

Plain Graph-Q-SAT **degrades** on large instances; GAT-Q-SAT holds up — the advantage **grows with size**.

Results — cross-domain transfer (no retraining)



Colouring-trained models, evaluated cold on *unseen* SATLIB domains (median MRIR, cap 200):

- **GAT-Q-SAT stays $\geq$ MiniSat on 3/4** domains (AIM, quasigroup, planning); plain Graph-Q-SAT on **1/4**.
- On the genuinely different domains (quasigroup, planning) **GAT beats MiniSat, Graph drops below**.
- Attention improves *cross-domain* generalisation, not just in-distribution fit.

Preliminary: AIM (72)/small-world (60) robust; quasigroup (11)/planning (8) small; single seed.

Where the field went & conclusions

- Lineage: NeuroSAT → NeuroCore → NeuroComb → **NeuroBack** (ICLR'24): a Graph *Transformer*, *offline* inference ⇒ first wall-clock speed-ups on Kissat — **validating the attention direction** (GAT-Q-SAT / GTv2-Q-SAT).
- **Our results**: graph attention helps on structured problems (colouring), *growing with size*, and improves **cross-domain robustness** (GAT $\geq$ MiniSat on 3/4 unseen domains vs 1/4 for Graph).
- The fixed-size CNN (AlphaZeroSAT) cannot generalise across sizes — the motivation for the graph methods.

Code, report and reproducible figures: `github.com/dmeoli/NeuroSAT`.

Thank you for your *attention*!

github.com/dmeoli/NeuroSAT